Complex Analysis II (MAT 528)

Date: May 24, 2026

Bernoulli Numbers

Author: Nolan R. H. Gagnon

I. Derivation of the Bernoulli Recurrence Relation

Let $f(z) = \frac{z}{e^z - 1}$. This function is analytic everywhere, except at $z = 0$. The singularity at $z = 0$ is, however, removable, since

$$\lim_{z\to 0} \frac{z}{e^z - 1} = \lim_{z\to 0} \frac{z}{\sum_{n=1}^{\infty} \frac{z^n}{n!}} \tag{1}$$

$$= \lim_{z\to 0} \frac{1}{\frac{\sum_{n=1}^{\infty} \frac{z^n}{n!}}{z}} \tag{2}$$

$$= \lim_{z\to 0} \frac{1}{\sum_{n=1}^{\infty} \frac{z^{n-1}}{n!}} \tag{3}$$

$$= \lim_{z\to 0} \frac{1}{1 + \sum_{n=2}^{\infty} \frac{z^{n-1}}{n!}} \tag{4}$$

$$= \frac{1}{1+0} \tag{5}$$

$$= 1. \tag{6}$$

Therefore, $f(z)$ can be expanded in a Taylor Series about the point $z_0 = 0$. That is, we can write

$$f(z) = \sum_{n=0}^{\infty} B_n \frac{z^n}{n!}.$$

The Bernoulli Numbers are defined to be the numbers in the sequence B_n, $n \geq 0$.

To find the Bernoulli Numbers, we first write $f(z) = \frac{z}{e^z - 1}$ as

$$f(z) = \frac{z}{\sum_{n=1}^{\infty} \frac{z^n}{n!}}.$$

Solving for z yields

$$z = f(z) \sum_{n=1}^{\infty} \frac{z^n}{n!} \tag{7}$$

$$= \left(\sum_{n=0}^{\infty} B_n \frac{z^n}{n!} \right) \left(\sum_{n=1}^{\infty} \frac{z^n}{n!} \right). \tag{8}$$

For simplicity's sake, let $C_n = \frac{B_n}{n!}$. Then (8) becomes

$$z = \left(\sum_{n=0}^{\infty} C_n z^n\right)\left(\sum_{n=1}^{\infty} \frac{z^n}{n!}\right) \tag{9}$$

$$= C_0 z + \left(\frac{C_0}{2!} + C_1\right) z^2 + \left(\frac{C_0}{3!} + \frac{C_1}{2!} + C_2\right) z^3 + \left(\frac{C_0}{4!} + \frac{C_1}{3!} + \frac{C_2}{2!} + C_3\right) z^4 + \ldots \tag{10}$$

From (10), we find that $C_0 = 1$ and the coefficients on the larger powers of z must be equal to 0. For example (and for clarification of the preceding sentence),

$$\frac{C_0}{3!} + \frac{C_1}{2!} + C_2 = 0.$$

Using this information, we find that

$$C_n = -\sum_{k=0}^{n-1} \frac{C_k}{(n-k+1)!}. \tag{11}$$

Thus, the Bernoulli Numbers can be found by

$$B_n = -n! \sum_{k=0}^{n-1} \frac{C_k}{(n-k+1)!}. \tag{12}$$

◀

II. Computing the Bernoulli Numbers

Two scripts were written to compute the first few Bernoulli Numbers, one in Python and the other in R. The Python script computes the numbers symbolically, which makes it possible to print them as rational numbers. The R script merely approximates the Bernoulli Numbers.

i. R Script

```
options(digits = 5)


## Function to compute first ncoefs Bernoulli Numbers
get_bernoulli <- function(ncoefs)
{
      ## Allocate storage for the desired Bernoulli Numbers
        B <- rep(NA, ncoefs)


        ## First Bernoulli number is 1
        B[1] <- 1


      ## Compute recursive sum
        for (n in 2 : ncoefs) {


                ## R indexes start at 1 instead of 0,
                ## so B[n] == 0 for n = 2k, k >= 4.
                if (n >= 4 && n%%2 == 0) {
                        B[n] <- 0
                } else {
                     ## Vector of factorials
                        fact_v <- factorial(seq(from = n, to = 2, by = -1))


                        ## Get previous (Bernoulli numbers) / n! needed to
                        ## Calculate the current one
                        prev_bern_v <- B[seq(from = 1, to = n - 1, by = 1)]


                        ## Calculate next (Bernoulli number) / n!
                        B[n] <- (-1) * sum(prev_bern_v / fact_v)
                }
        }
```

```
        ## Multiply by n! to get actual Bernoulli numbers
        B <- B * factorial(seq(from = 0, to = ncoefs - 1, by = 1))
        return(B)
}

## Print first 100 Bernoulli Numbers
bernoulli_vect = get_bernoulli(100)
print(bernoulli_vect)
```

ii. R Output (first 50 Bernoulli Numbers)

```
[1]  1.0000e+00 -5.0000e-01  1.6667e-01  0.0000e+00 -3.3333e-02  0.0000e+00
[7]  2.3810e-02  0.0000e+00 -3.3333e-02  0.0000e+00  7.5758e-02  0.0000e+00
[13] -2.5311e-01  0.0000e+00  1.1667e+00  0.0000e+00 -7.0922e+00  0.0000e+00
[19]  5.4971e+01  0.0000e+00 -5.2912e+02  0.0000e+00  6.1921e+03  0.0000e+00
[25] -8.6580e+04  0.0000e+00  1.4255e+06  0.0000e+00 -2.7298e+07  0.0000e+00
[31]  6.0158e+08  0.0000e+00 -1.5116e+10  0.0000e+00  4.2962e+11  0.0000e+00
[37] -1.3712e+13  0.0000e+00  4.8836e+14  0.0000e+00 -1.9301e+16  0.0000e+00
[43]  8.4245e+17  0.0000e+00 -4.0484e+19  0.0000e+00  2.1457e+21  0.0000e+00
[49] -1.2786e+23  0.0000e+00
```

iii. Python Script

```
from sympy import Rational     ## For doing math with fractions symbolically
import numpy as np             ## For vectorizing computations
from scipy.misc import factorial ## Vectorized factorial function



##
## Brief:      Calculates the first ncoefs Bernoulli numbers.
## Param ncoefs: Number of Bernoulli numbers to compute.
## Returns:    First ncoefs Bernoulli numbers (as Rationals)
##             in a numpy array.
##
def get_bernoulli(ncoefs):

        ## Storage for Bernoulli numbers
        B = []

        ## Place the first Bernoulli number (B_0 = 1) in storage
        B.append(Rational(1, 1))

        for n in range(1, ncoefs):

                ## For n = 2k + 1 (k > 0), B_n = 0
                if ((n > 1) and (n % 2 != 0)):

                        B.append(Rational(0, 1))

                else:
                        ## Create array of the form [1/(n+1)!, 1/n!, ..., 1/3!, 1/2!]
                        fact_arr = np.array([Rational(1, int(factorial(i))) for i in range(n + 1, 1, -1)])

                        ## Create array of the form [B_0, B_1, ..., B_(n-1)]
                        prev_bern_arr = np.array([B[i] for i in range(0, n)])

                        ## Calculate the next Bernoulli number using vectorized math
```

```
                    next_B_num = (-1) * sum(np.multiply(prev_bern_arr, fact_arr))

                    ## Store the calculated Bernoulli number
                    B.append(next_B_num)

        ## Convert the list B into a numpy array
        B = np.array(B)

        ## Create numpy array of the form [0!, 1!, 2!, ..., (ncoefs-1)!]
        f = np.array([Rational(int(factorial(i)), 1) for i in range(0, ncoefs)])

        ## Vectorized-multiply B and f
        B = np.multiply(B, f)

        ## Return array of Bernoulli numbers to caller
        return B


## Number of Bernoulli numbers to print
num_bern = 22

## Get first num_bern Bernoulli numbers
B = get_bernoulli(num_bern)

## Print Bernoulli numbers to screen
print("The first %d Bernoulli numbers are as follow: \n" % num_bern)
for n in range(0, num_bern):
    print("B_%d:" % n + " " + str(B[n]))
```

---

iv. Python Output (First 22 Bernoulli Numbers)

```
The first 22 Bernoulli numbers are as follow:

B_0: 1  B_1: -1/2  B_2: 1/6  B_3: 0 B_4: -1/30  B_5: 0  B_6: 1/42  B_7: 0
B_8: -1/30  B_9: 0  B_10: 5/66  B_11: 0 B_12: -691/2730  B_13: 0  B_14: 7/6
B_15: 0  B_16: -3617/510 B_17: 0 B_18: 43867/798 B_19: 0 B_20: -174611/330
B_21: 0
```